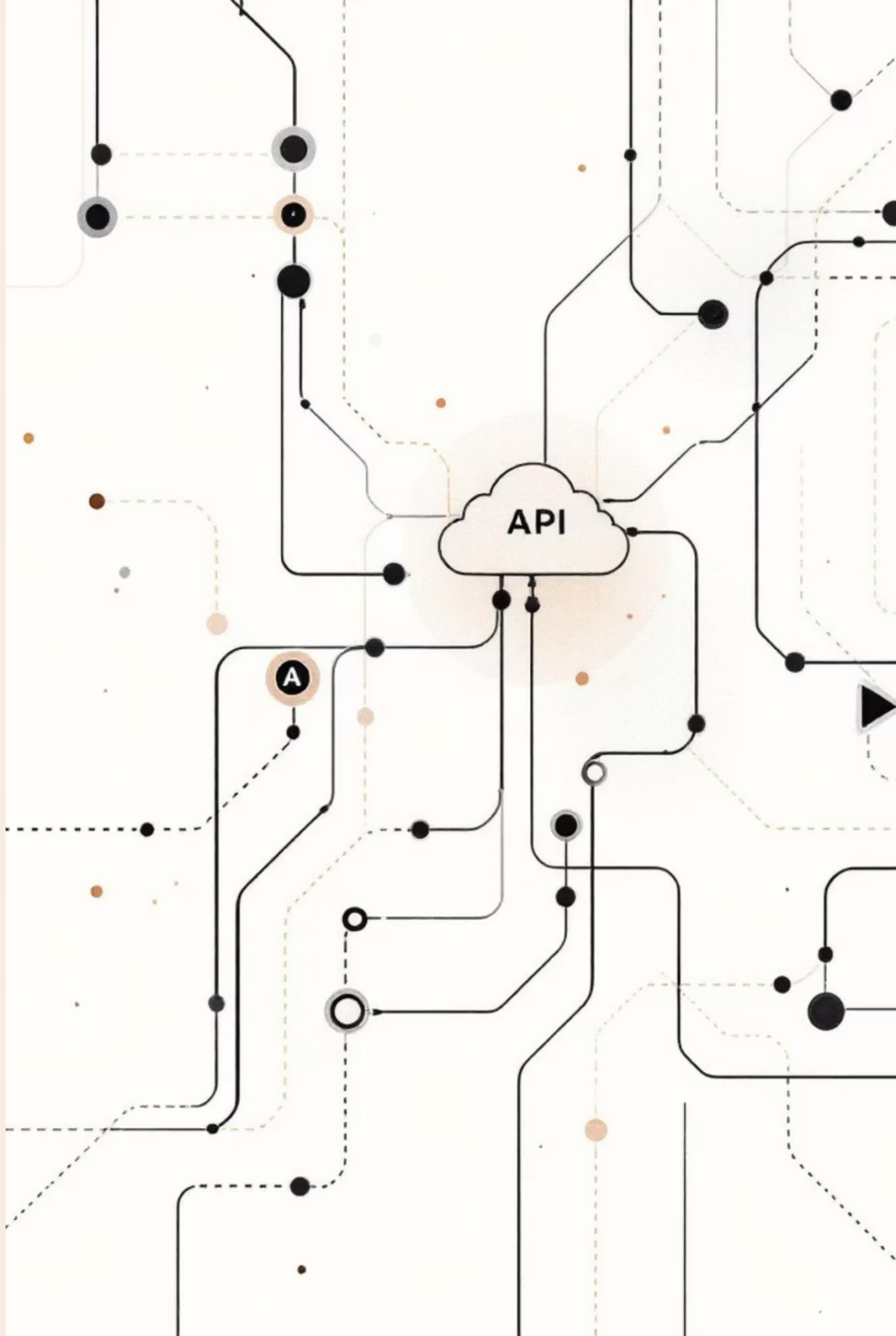
API Test Generator

DevOps Automation Tool

Student: Nemat Sachdeva | **Reg. No:** 23FE10CSE00107

Course: CSE3253 DevOps | **Semester:** VI

Made under guidance of: Dr. Dibakar Sinha



Problem Statement

The Challenge

Modern applications rely heavily on REST APIs for communication between services. As API complexity grows, ensuring reliability becomes critical.

Writing API tests manually is time-consuming, repetitive, and error-prone. Developers spend valuable time writing boilerplate test code instead of focusing on business logic.

1

Manual Testing

Time-consuming and repetitive

2

Need for Automation

Fast, reliable test generation

3

Scalable Solution

Handle complex API specifications

Project Objectives

This project aims to automate API test generation by leveraging OpenAPI/Swagger specifications.

01

Parse OpenAPI/Swagger Specifications

Extract API definitions from YAML or JSON format

02

Extract API Endpoints

Automatically identify all available endpoints, methods, and parameters

03

Generate pytest Test Cases

Create comprehensive, executable test suites with assertions

04

Integrate with CI/CD Pipeline

Enable automated testing on every code commit

05

Enable Automated API Testing

Validate API functionality automatically without manual intervention

System Architecture

Swagger/OpenAPI File

YAML or JSON specification defining API endpoints, methods, parameters, and responses

Swagger Parser

Python-based parser that reads and interprets the OpenAPI specification structure

API Test Generator

Core engine that transforms API definitions into pytest test cases with assertions

Generated pytest Test Suite

Executable Python test files ready to validate API functionality

CI/CD Pipeline

Automated execution and reporting in development workflow

Technology Stack

Backend

Python - Core language

Flask - Web framework

Testing

pytest - Test framework

requests - HTTP client

DevOps

Docker - Containerization

GitHub Actions - CI/CD

Kubernetes - Orchestration

Cloud

Railway - Backend hosting

Vercel - Frontend hosting

Monitoring

Prometheus - Metrics

Nagios - Alerts

Grafana - Visualization

Key Features

- **Parse OpenAPI and Swagger specifications**

  Support for both YAML and JSON formats

- **Extract API endpoints automatically**

  Identify all routes, methods, and parameters

- **Generate automated pytest test cases**

  Create complete test suites with assertions

- **Web interface for uploading API specifications**

  User-friendly interface for easy adoption

- **Docker containerized application**

  Consistent environment across deployments

- **CI/CD automation**

  Integrated with GitHub Actions workflow

- **Cloud deployment support**

  Deploy to Railway and Vercel platforms

DevOps Pipeline

Continuous Integration & Deployment Workflow

1. **Code Commit**
   Developer pushes changes to GitHub repository
2. **CI Pipeline Trigger**
   GitHub Actions automatically starts pipeline execution
3. **Lint & Test**
   Code quality checks and test suite execution
4. **Docker Build**
   Container image creation and tagging
5. **Deployment**
   Application deployed to production environment

Continuous Testing

Automated tests run on every commit

Automated Builds

No manual intervention required

Faster Deployment

Reduce time from code to production

Deployment Architecture

Separate frontend and backend components for scalability and maintainability.

Frontend

React.js application hosted on **Vercel**

Provides user interface for uploading specifications

Backend

Flask API hosted on **Railway**

Handles file parsing and test generation

CI/CD

GitHub Actions manages automation

Orchestrates builds and deployments

Containerization

Docker ensures consistency

Same environment everywhere

Workflow Demonstration

How the tool works in practice:

Upload Specification

User uploads Swagger or OpenAPI file through web interface

Parse Endpoints

System analyzes API structure and extracts all endpoints, methods, and parameters

Generate Tests

Tool creates complete pytest test cases with assertions for each endpoint

Download & Run

User downloads test suite or runs tests directly in environment

Validate API

API functionality is automatically validated against specifications

Future Improvements & Conclusion

Future Enhancements

Smart Test Data Generation

Automatically create realistic test data based on parameter types and constraints

Dynamic Parameter Handling

Support complex parameter dependencies and dynamic values in requests

Response Schema Validation

Add automatic schema validation to verify response structures match specifications

Advanced API Mocking

Generate mock servers from specifications for testing without real endpoints

Full UI Automation

Extend to generate frontend integration tests alongside API tests

---

Conclusion

This project demonstrates DevOps automation by combining API parsing, automated test generation, CI/CD pipelines, containerization, and cloud deployment into a cohesive solution.